

# Statistics with Recitation: TA Session

Danny Po-Hsien Kang (康柏賢)

October 7, 2025

# Today's agenda

- 1 Random Number Generator
  - `set.seed()`
  - `runif()`
  - `rnorm()`
  - `rbinom()`
  - `rpois()`
  - `rexp()`
- 2 Data Wrangling: Combine Dataset
  - `rbind()`
  - `cbind()`
- 3 Standardization
  - `scale()`
- 4 Multiple Testing
  - `linearHypothesis()`

# Today's Dataset

- Today we will use the following CSV file from the OpenIntro Website:
  - `ncbirths.csv`: A 2004 dataset from North Carolina on births, including mothers' habits and practices, with 1,000 cases.
- Please import the following datasets into RStudio.

```
birth <- read.csv("data/ncbirths.csv")
```

# Set Random Seed: `set.seed()`

- `set.seed()` sets R's random-number generator (RNG) state
  - Random sampling functions: `runif()`, `rnorm()`, `sample()`
  - Without setting seeds, we will obtain different outcomes every time we use these functions.
  - Key to **reproducibility**: It guarantees that the code produces the same sequence every time you rerun.
- **Syntax:**

```
set.seed(seed)
```

- **Example:**

```
set.seed(20260407)
```

# Generate Uniform Random Samples: `runif()`

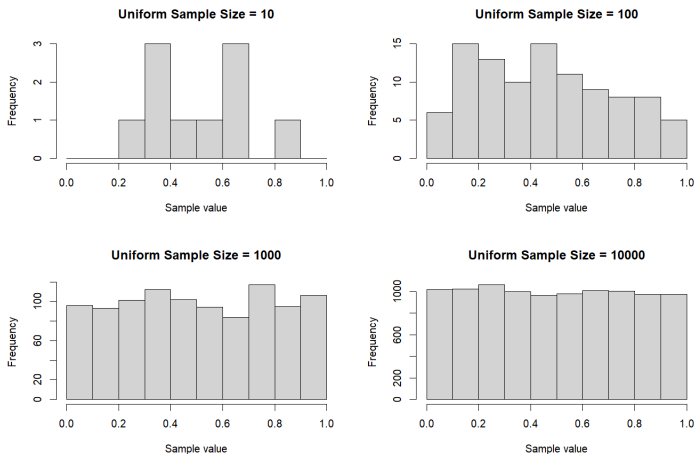


Figure: Random samples drawn from  $\text{Uniform}[0,1]$  in different sizes

# Generate Uniform Random Samples: runif()

- **Syntax:**

```
num <- runif(n = ..., min = ..., max = ...)
```

- **Example:**

```
rdnum <- runif(10) # min = 0, max = 1  
rdnum_2 <- runif(n = 10, min = 0, max = 100)
```

- **Output**

```
print(rdnum)  
[1] 0.7996254 0.7107656 0.4796840 0.6867655 0.7798512  
[6] 0.6304974 0.6980979 0.8342457 0.5835580 0.9687111  
  
print(rdnum_2)  
[1] 0.7996254 0.7107656 0.4796840 0.6867655 0.7798512  
[6] 0.6304974 0.6980979 0.8342457 0.5835580 0.9687111
```

# Generate Normal Random Samples: `rnorm()`

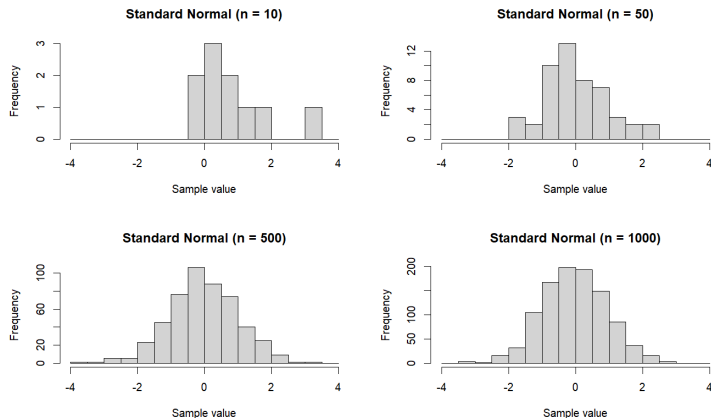


Figure: Random samples drawn from  $\mathcal{N}(0, 1)$  in different sizes

# Generate Normal Random Samples: rnorm()

- **Syntax:**

```
num <- rnorm(n = ..., mean = ..., sd = ...)
```

- **Example:**

```
rdnum <- rnorm(10) # mean = 0, sd = 1  
rdnum_2 <- rnorm(n = 10, mean = 60, sd = 15)
```

- **Output**

```
print(rdnum)  
[1]  0.836085280 -0.630010803 -0.001762710  2.297059910 -0.8165  
[6] -2.015860275  0.254655772  0.108503743  0.004288487  0.4861  
  
print(rdnum_2)  
[1] 66.06636 71.63407 75.30722 60.50602 46.48625  
[6] 52.42269 67.73190 41.56804 72.80670 61.29849
```



# Generate Binomial Random Samples: `rbinom()`

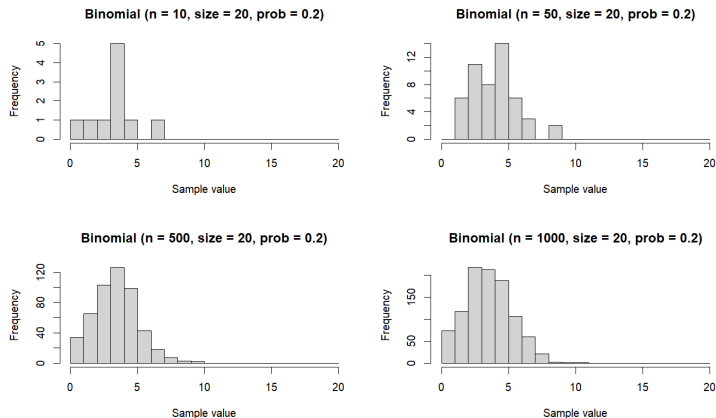


Figure: Random samples drawn from  $\text{Bin}(20, 0.2)$  in different sizes

# Generate Binomial Random Samples: rbinom()

- **Syntax:**

```
a <- rbinom(n = ..., size = ..., prob = ...)
```

- **Example:**

```
rdnum <- rbinom(n = 30, size = 20, prob = .5)
```

- **Output**

```
print(rdnum)
[1] 10 11  9 11 13  8 11 11  8 12
[11]  8  7 13 10  8 13 12 11 11 10
[21] 10  7 10 10 10 15  8  7  8 11

print(mean(rdnum))
[1] 10.1
```

# Generate Poisson Random Samples: `rpois()`

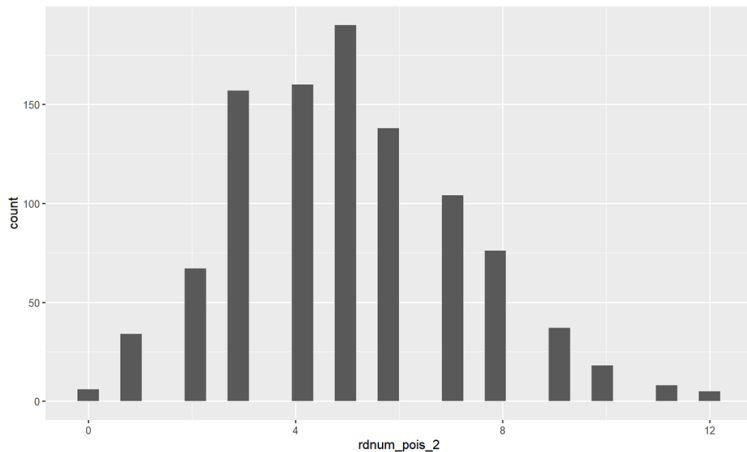


Figure: Random samples with size 1,000 drawn from  $\text{Poisson}(\lambda = 5)$

# Generate Poisson Random Samples: `rpois()`

- **Syntax:**

```
num <- rpois(n = ..., lambda = ...)
```

- **Example:**

```
rdnum_pois <- rpois(20, lambda = 5)
```

- **Output:**

```
> print(rdnum_pois)
[1] 3 4 5 1 3 7 8 4 5 10 8 6 3 11 1 4 4 3 7 4
```

# Generate Exponential Random Samples: `rexp()`

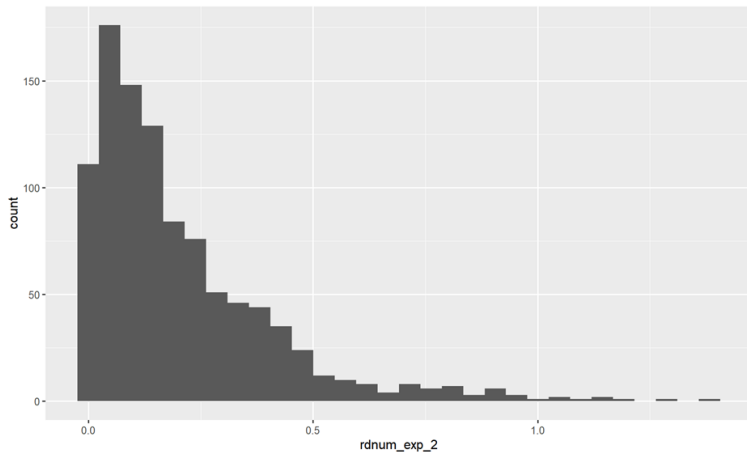


Figure: Random samples with size 1,000 drawn from  $\text{Exp}(\lambda = 5)$

# Generate Exponential Random Samples: rexp()

- **Syntax:**

```
num <- rexp(n = ..., rate = ...)
```

- **Example:**

```
rdnum_exp <- rexp(20, rate = 5)
```

- **Output:**

```
> print(rdnum_exp)
[1] 0.032601 0.053080 0.219614 0.615566 0.047792
[6] 0.112196 0.194019 0.119259 0.960832 0.462925
[11] 0.113932 0.305901 0.493771 0.236433 0.018397
[16] 0.111395 0.292260 0.293656 0.203214 0.138105
```

# Two Ways to Combine Your Dataset

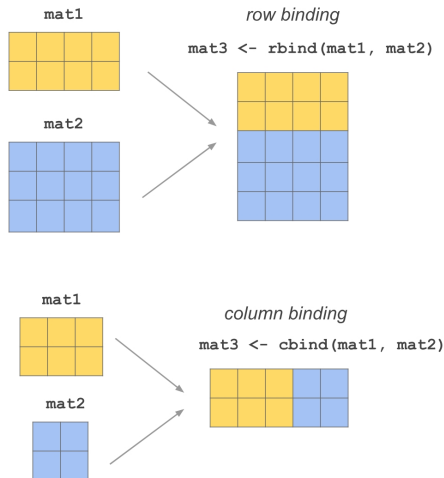


Figure: Combine dataset by rows or columns (Gaston Sanchez, 2024)

# Combine Dataset by Rows: rbind()

- **Example:**

```
## first, let's draw two different sets of random sample
rdnum_pois_20 <- rpois(n = 20, lambda = 5)
rdnum_pois_10000 <- rpois(n = 10000, lambda = 5)

## using the two sets of random sample, build two DataFrames
df_20 <- data.frame(
  size = 20,
  values = rdnum_pois_20
)
df_10000 <- data.frame(
  size = 10000,
  values = rdnum_pois_10000
)

## combine the two DataFrames together, and draw the histogram
df <- rbind(df_20, df_10000)
```



# Combine Dataset by Rows: rbind()

- **Example:**

```
ggplot(df, aes(x = values)) +  
  geom_histogram() +  
  facet_wrap(~size, scales = "free_y")
```

# Combine Dataset by Rows: rbind()

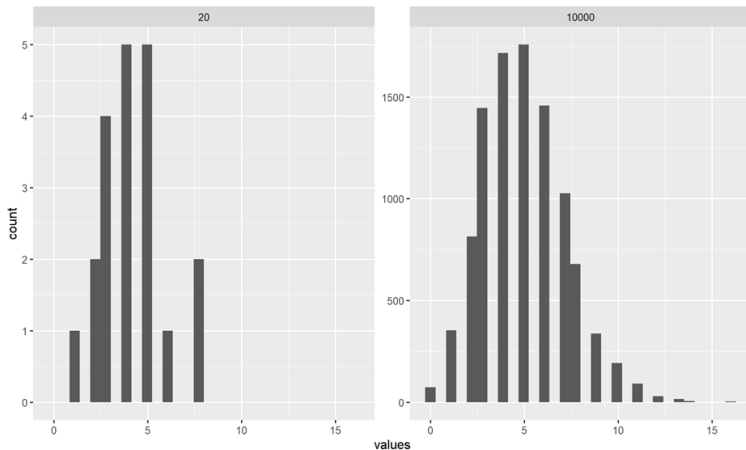


Figure: Histogram of two samples in different sample sizes

# Combine Dataset by Columns: cbind()

- **Example:**

```
## first, let's draw two different sets of random sample
rdnum_pois_param5 <- rpois(n = 1000, lambda = 5)
rdnum_pois_param20 <- rpois(n = 1000, lambda = 20)

## using the two sets of random sample, build two DataFrames re
df_param5 <- data.frame(
  values_param5 = rdnum_pois_param5
)
df_param20 <- data.frame(
  values_param20 = rdnum_pois_param20
)
```

# Combine Dataset by Columns: cbind()

- **Example:**

```
df <- cbind(df_param5, df_param20)

ggplot(df) +
  geom_histogram(mapping = aes(x = values_param5,
                              fill = "lambda = 5"),
                alpha = 0.7
  ) +
  geom_histogram(mapping = aes(x = values_param20,
                              fill = "lambda = 20"),
                alpha = 0.7
  )
```

# Combine Dataset by Columns: `cbind()`

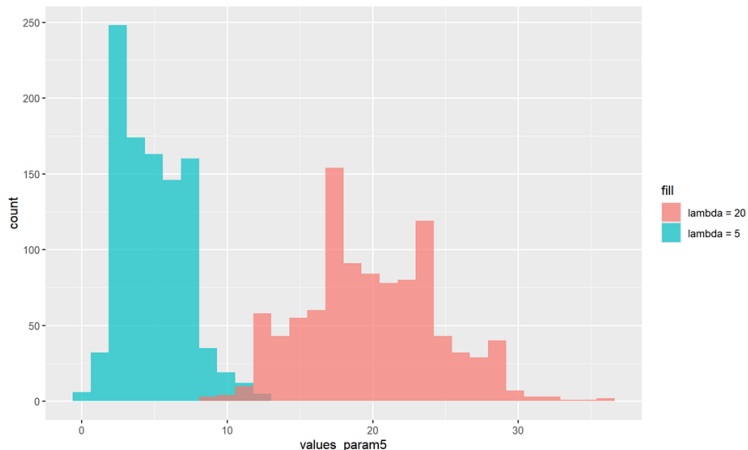


Figure: Histogram of two samples in different distribution parameters

# Standardize Variables: scale()

- **Syntax:**

```
new_var <- scale(x = ..., center = TRUE, scale = TRUE)
```

- **Example:**

```
score <- c(70, 80, 75, 95)  
scale(score)
```

- **Output:**

```
      [,1]  
[1,] -0.9258201  
[2,]  0.0000000  
[3,] -0.4629100  
[4,]  1.3887301  
attr(,"scaled:center")  
[1] 80  
attr(,"scaled:scale")  
[1] 10.80123
```

# Standardize Variables: scale()

- If center = TRUE, then scale() subtracts the mean:  $x_i - \bar{x}$
- If scale = TRUE, then scale() divides by the standard deviation  $\frac{x_i}{s_x}$
- So when center = TRUE and scale = TRUE,  $z_i = \frac{x_i - \bar{x}}{s_x}$
- **Example:**

```
scale(score, center = FALSE, scale = TRUE)
```

- **Output:**

```
      [,1]  
[1,] 0.7526447  
[2,] 0.8601653  
[3,] 0.8064050  
[4,] 1.0214463  
attr(,"scaled:scale")  
[1] 93.00538
```

# Multiple Testing: linearHypothesis()

- Install and load the package:

```
install.packages("car")  
library(car)
```

- **car**: Short for Companion to Applied Regression.
- We will use this package to conduct multiple hypothesis testing.



# Multiple Testing: `linearHypothesis()`

- Consider the following model:

$$\text{Weight}_i = \beta_0 + \beta_1 \text{Week}_i + \beta_2 \text{Gender}_i + \beta_3 \text{Visit}_i + \varepsilon_i$$

- We use the `lm_robust()` function to estimate the model.

```
lm_model_multiple_robust <- lm_robust(  
  data = birth,  
  weight ~ weeks + gender + visits  
)
```

# Multiple Testing: `linearHypothesis()`

- Expected output:

|             | Estimate     | Std. Error  | t value    | Pr(> t )      |
|-------------|--------------|-------------|------------|---------------|
| (Intercept) | -6.208828047 | 0.532881547 | -11.651423 | 1.745020e-29  |
| weeks       | 0.339617332  | 0.013969817 | 24.310794  | 1.162998e-102 |
| gendermale  | 0.365472826  | 0.070476825 | 5.185716   | 2.610867e-07  |
| visits      | 0.009322152  | 0.009229467 | 1.010042   | 3.127228e-01  |

- Single hypothesis testing results for each coefficient can be directly checked from the regression output:
  - standard error
  - t-value
  - p-value
- To test multiple coefficients simultaneously (joint hypothesis), we need to use the `linearHypothesis()` function.

# Multiple Testing: linearHypothesis()

- **Syntax:**

```
library(car) # Companion to Applied Regression

linearHypothesis(
  Linear_Model,
  c("Hypothesis 1", "Hypothesis 2", ...),
  test = "F"
)
```

- **Example:**

```
linearHypothesis(
  lm_model_multiple_robust,
  c("weeks = 0", "visits = 0"), # or c("weeks = visits")
  test = "F", # or Chisq
)
```

# Multiple Regression: Multiple Testing

## • Output:

```
Linear hypothesis test:
```

```
weeks = 0
```

```
visits = 0
```

```
Model 1: restricted model
```

```
Model 2: weight ~ weeks + gender + visits
```

|   | Res.Df | Df | F      | Pr(>F)        |
|---|--------|----|--------|---------------|
| 1 | 988    |    |        |               |
| 2 | 986    | 2  | 309.37 | < 2.2e-16 *** |

```
---
```

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1
```